

普及组初赛模拟题（五）

一、单项选择题（共15题，每题2分，共计30分；每题有且仅有一个正确选项）

1. *IPv4* 协议使用 32 位地址，随着其不断被分配，地址资源日趋枯竭，因此，它正逐渐被 *IPv6* 协议所取代，*IPv6* 使用 128 位地址，则 1 个 *IPv6* 所占的字节是：
 - A. 16
 - B. 8
 - C. 12
 - D. 20
2. 将一组数 1, 3, 4, 7, 9 依次存放入数组 a 的下标为 1, 2, 3, 4, 5 单元格中，则 $a[a[a[2]]+1]$ 表达式的值是：
 - A. 3
 - B. 5
 - C. 7
 - D. 9
3. 有 5 个一模一样的苹果，放到 3 个一样的盘子中，允许盘子不放苹果，一共有（ ）种放法。
 - A. 4
 - B. 5
 - C. 10
 - D. 11
4. 假设一棵二叉树的先序遍历序列为 52134869，中序遍历序列为 12345689，则其后序遍历序列为()。
 - A. 14326985
 - B. 13426985
 - C. 96843125
 - D. 12436895
5. 在 $C++$ 程序中，表达式 $200|10$ 的值是（ ）。
 - A. 20
 - B. 202
 - C. 1
 - D. 220
6. 定义一颗有根树的深度:根结点的深度为 0，其余结点的深度等于该结点的父亲结点的深度加 1。以下数字中哪一个可以作为一颗深度为 9 的满二叉树的总节点数()。

- A. 511
- B. 516
- C. 1023
- D. 1026

7. 7 个同学围坐一圈，要选 2 个不相邻的作为代表，有()种不同的选法。

- A. 12
- B. 14
- C. 16
- D. 18

8. 下面的故事与（ ）法有着异曲同工之妙：

从前有座山，山里有座庙，庙里有个老和尚在给小和尚讲故事：从前有座山，山里有座庙，庙里有个老和尚在给小和尚讲故事：从前有座山，山里有座庙，庙里有个老和尚给小和尚讲故事.....

- A. 枚举
- B. 贪心
- C. 递归
- D. 分治

9. 当 $A \geq B \ \&\& \ B \geq C$ 的取值为真时，表达式 $A > C \mid \mid B == C$ 的值()。

- A. 为真
- B. 无法判定结果的真假
- C. 只有当 A、B、C 都相等时才为真
- D. 也有可能为假

10. 设 G 是有 6 个结点的完全图，要得到一颗生成树，需要从 G 中删去（ ）条边。

- A. 6
- B. 9
- C. 10
- D. 15

11. 设一组初始记录关键字序列为 $\{50, 40, 95, 20, 15, 70, 60, 45\}$ ，进行排序，每一次操作将未排序部分最小值和该部分的首元素对调，在排序过程中，前 4 个元素在某个时刻可能为（ ）。

- A. 15, 20, 50, 40
- B. 15, 40, 60, 20
- C. 15, 20, 40, 50
- D. 15, 20, 40, 45

12. 给定 m 种颜色和有 n 个点的手环，要求用这个 m 种颜色给这条手环染色，其中旋转和翻转能够互相得到的算同一种染色方案，如对于 3 个点的手环， ABC 的染色方法与 BCA (旋转)、 ACB (翻转)是相同的。当 $m = 2, n = 2$ 时，一共有 3 种染色方案，分别为 AA、AB、BB，那么当 $m = 5, n = 4$ 时，一共有()种染色方案。

- A. 60
- B. 120

◦ C. 160

◦ D. 625

13. 给出以下邻接矩阵，其表示的图是 *DAG* (有向无环图)的是()。

◦ A.

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

◦ B.

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

◦ C.

$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

◦ D.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

14. 设一组初始记录关键字序列为 (25, 50, 15, 35, 80, 85, 20, 40, 36, 70)，其中含有 5 个长度为 2 的有序字表，则用归并排序的方法对该记录关键字序列进行一趟归并排序后的结果为 ()。

◦ A. 15, 25, 35, 50, 20, 40, 80, 85, 36, 70

◦ B. 15, 25, 35, 50, 80, 20, 85, 40, 70, 36

◦ C. 15, 25, 35, 50, 80, 85, 20, 36, 40, 70

◦ D. 15, 25, 35, 50, 80, 20, 36, 40, 70, 85

15. 栈 *S* 最多能容纳 4 个元素。现有 6 个元素按 1, 2, 3, 4, 5, 6 的顺序进栈，问下列哪一个序列是可能的出栈序列()。

◦ A. 1, 4, 6, 5, 2, 3

◦ B. 2, 3, 5, 6, 1, 4

◦ C. 3, 2, 5, 4, 1, 6

◦ D. 5, 4, 3, 2, 1, 6

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 T，错误填 F；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

注意：判断题正确填 T，错误填 F。

阅读下面程序，完成第 16 ~ 21 题。

```
1  #include<iostream>
2  #include<cstdio>
3  using namespace std;
4  int i, j, n;
5  int x[101], y[101];
6  int main() {
7      cin >> n;
8      for (i = 1; i <= n; i++) cin >> x[i];
9      for (i = 1; i < n; i++)
10         for (j = i + 1; j <= n; j++)
11             if (x[i] > x[j])
12                 y[j]++;
13             else if (x[i] < x[j])
14                 y[i]++;
15     for (i = 1; i <= n; i++)
16         printf("%d ", y[i]);
17     cout << endl;
18     return 0;
19 }
```

16. 把第 12 行与第 14 行互换位置，结果不会改变。[]
17. 第 13 行把 `if(x[i] < x[j])` 删掉效果一样。[]
18. (2分) 数组 $y[i]$ 中存的是 $x[i]$ 在数列中从大到小的次序。[]
19. (2分) 第 10 行把 $i + 1$ 改成 1，数组 y 每个元素的值增加 1 倍。[]
20. 此程序如果 n 输入 4，然后输入 2 4 1 3，输出结果是()。
- A. 4 3 2 1
 - B. 1 2 3 4
 - C. 2 0 3 1
 - D. 1 3 0 2
21. 此程序的时间复杂度是()。
- A. $O(\log n)$
 - B. $O(n^2)$
 - C. $O(n \log n)$
 - D. $O(n)$

阅读下面程序，完成第 22 ~ 27 题。

```
1  #include<iostream>
2  #include<cstdio>
3  using namespace std;
4  int j, i, m;
5  int a[10];
6  int main() {
7      for (i = 2; i <= 6; i++)
8          a[i] = i + 1;
9      do {
10         m = 2;
11         for (i = 3; i <= 6; i++)
12             if (a[m] > a[i]) m = i;
13         a[m] = a[m] + m;
14         m = 1;
15         for (i = 2; i <= 5; i++)
16             for (j = i + 1; j <= 6; j++)
17                 if (a[i] < a[j]) m = 0;
18     } while (m == 0);
19     printf("%d", a[2]);
20     return 0;
21 }
```

22. 程序结束时， $a[2]$ 的值一定是数组 a 中的最大值。[]
23. 第 18 行 $m==0$ 成立时，数组 $a[i]$ ($2 \leq i \leq 6$) 从大到小排序 []
24. 程序输出时， a 数组满足:对任意的 $2 \leq i < 6$ ，有 $a[i] > a[i + 1]$ 。[]
25. 删除第 14 行代码 $m=1$ 程序结果会发生改变。[]
26. 程序的输出结果为()
- A. 58
 - B. 61
 - C. 59
 - D. 60
27. (4分) 第 17 行 $\text{if} (a[i] < a[j])$ 执行了多少次()。
- A. 800
 - B. 820
 - C. 10
 - D. 30

阅读下面程序，完成第 28 ~ 33 题。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  vector<int> g[30];
4  struct node {
5      int u, v, len;
6  } ed[100];
7  int maxx = -1, vis[100];
8  void dfs(int u, int len) {
9      maxx = max(maxx, len);
10     for (int i = 0; i < g[u].size(); i++) {
11         int v = g[u][i];
12         if (!vis[v]) {
13             vis[v] = 1;
14             dfs(v, len + ed[v].len);
15             vis[v] = 0;
16         }
17     }
18 }
19 int main() {
20     int n;
21     cin >> n;
22     for (int i = 1; i <= n; i++) {
23         string str;
24         cin >> str;
25         ed[i].u = str[0] - 'a';
26         ed[i].v = str[str.size() - 1] - 'a';
27         ed[i].len = str.size();
28         g[ed[i].u].push_back(i);
29     }
30     for (int i = 1; i <= n; i++) {
31         memset(vis, 0, sizeof(vis));
32         vis[i] = 1;
33         dfs(i, ed[i].len);
34     }
35     cout << maxx << endl;
36     return 0;
37 }

```

数据保证： 输入的字符串中只包含小写英文字母。

28. 如果输入 $n = 100$ ，程序可以正常运行，且输出结果正确。[]
29. (2分) 程序中如果将 vis 标记数组删除，不影响最终的答案。[]
30. 删除第 31 行 `memset(vis, 0, sizeof(vis));`，在第 33 行下方补充 `vis[i] = 0;` 后，不影响最终的答案。[]
31. 程序的功能是：输入 n 个字符串，如果两个字符串满足：第一个字符串的末尾字符和第二个字符串的开头字符相同，则这两个字符串可以进行拼接，求从 n 个字符串中挑选若干字符串进行拼接时的可行性方案数。[]

32. (4分) 若输入：

```
4
jisuankec
csp
come
on
```

则输出：[]

- A. 9
- B. 12
- C. 13
- D. 18

33. 请根据程序，选出正确的选项。[]

- A. 对于每个字符串，会建立一条无向边
- B. 程序中是使用邻接矩阵存储图
- C. `void dfs(int u, int len)` 其中 u 表示的是第 u 个字符串构成边的编号
- D. `void dfs(int u, int len)` 其中 len 表示当前选择字符串的长度和

三、完善程序（单选题，每小题3分，共计30分）

阅读下面题目，完成第 34 ~ 38 题。

现在政府计划在某个区域的城市之间建立高速公路，以使得其中任意两个城市之间都有直接或间接的高速公路相连。费用为每千米为一个单位价格，求最小费用。

输入： n ($n \leq 100$ ，表示城市数目)。

接下来 n 行，每行两个数 x_i, y_i ，表示第个城市的坐标，(单位:千米)。

输出：最小费用(保留 2 位小数)。

程序如下：

```

1  #include<iostream>
2
3  #include<cstdio>
4
5  using namespace std;
6  const int maxn = 101;
7  struct tcity {
8      float x, y;
9  };
10 tcity c[maxn];
11 float d[maxn][maxn], a, minf;
12 int p[maxn], n, i, j, k;
13 int main() {
14     cin >> n;
15     for (i = 1; i <= n; i++)
16         cin >> c[i].x >> c[i].y;
17     for (i = 1; i <= n; i++)
18         for (j = i; j <= n; j++)
19             d[i][j] = ____(1) ____;
20     p[1] = 0;
21     for (i = 2; i <= n; i++) ____(2) ____;
22     for (i = 1; i <= n - 1; i++) {
23         minf = 1E10;
24         for (j = 1; j <= n; j++) {
25             if (__(3) ____) {
26                 minf = d[p[j]][j];
27                 (__(4) ____);
28             }
29         }
30         a = a + d[p[k]][k];
31         p[k] = 0;
32         for (j = 1; j <= n; j++)
33             if (__(5) ____) p[j] = k;
34     }
35     printf("%.2f", a);
36     return 0;
37 }

```

34. ① 处应填()

- A. $\sqrt{(c[i].x-c[i].y)*(c[j].x-c[j].y)+(c[i].x-c[i].y)*(c[j].x-[j].y))}$
- B. $\sqrt{(c[i].x-c[i].y)*(c[i].x-c[i].y)+(c[j].x-c[j].y)*(c[i].x-[j].y))}$;
- C. $\sqrt{(c[i].x-c[j].x)*(c[j].x-c[i].x)+(c[i].y-c[j].y)*(c[j].y-[i].y))}$;
- D. $\sqrt{(c[i].x-c[j].x)*(c[i].x-c[j].x)+(c[i].y-c[j].y)*(c[i].y-[j].y))}$;

35. ② 处应填()

- A. $p[i]=1$;
- B. $p[i]=0$;
- C. $p[n-i]=0$;

- D. `p[n-i]=0;`

36. ③ 处应填()

- A. `p[j]==0||d[p[j]][j]<minf`
- B. `p[j]!=0&& d[p[j]][j]<minf`
- C. `p[i]==0&& d[p[j]][j]< minf`
- D. `p[i]!=0||d[p[j]][j]<minf`

37. ④ 处应填()

- A. `k=0;`
- B. `k=i;`
- C. `k=minf;`
- D. `k=j;`

38. ⑤ 处应填()

- A. `d[p[j]][j]>d[k][j]`
- B. `d[p[j]][j]<d[k][j]`
- C. `d[p[i]][j]>d[k][j]`
- D. `d[p[i]][j]<d[k][j]`

阅读下面题目，完成第 39 ~ 43 题。

问题描述

欧拉回路问题由七桥问题而来，其基本问题是是否能一次性不重复地走遍这七座桥，转换为数学问题中的图论就是指是从图中的一个顶点出发，是否能够一次性不回头地走遍所有的边，算法代码如下：

```

1  #include <iostream>
2  #include <ctime>
3  using namespace std;
4  int G[5][5];
5  int visited[5][5];
6  int n = 5;
7  void euler(int u) {
8      for (int v = 0; v < n; v++) {
9          if (G[u][v] && ____ (1) ____ ) {
10             cout << u << "->" << v << endl;
11             visited[u][v] = visited[v][u] = ____ (2) ____ ;
12             ____ (3) ____
13         }
14     }
15 }
16
17 int main() {
18     G[1][2] = G[2][1] = G[1][3] = ____ (4) ____ = 1;
19     G[2][4] = G[4][2] = G[3][4] = ____ (5) ____ = 1;
20     euler(1);
21     return 0;
22 }

```

39. ① 处应填()

- A. !visited[u][v]
- B. visited[u][v]
- C. visited[v][u]
- D. G[v][u]

40. ② 处应填()

- A. 0
- B. 1
- C. u
- D. v

41. ③ 处应填()

- A. G[u][v] = 0;
- B. euler(v);
- C. G[v][u] = 0;
- D. euler(u);

42. ④ 处应填()

- A. G[1][0]
- B. G[0][1]
- C. G[0][3]
- D. G[3][1]

43. ⑤ 处应填()

- A. `G[2][1]`
- B. `G[2][0]`
- C. `G[0][2]`
- D. `G[4][3]`